

Study protocol - synthetic NSCLC-inspired imaging phenotype model

Protocol version 2.0 | 25 September 2026 | Repository: invariant-demo-nsclc | Dataset: invariant-synthetic-nsclc-v1, generator 1.1.0

1. Purpose and status

This protocol specifies how a small image classifier is trained and evaluated on a fully synthetic, NSCLC-inspired benchmark, and which invariants every evaluation must satisfy. The prediction target is a simulated binary imaging phenotype. It is not cancer diagnosis, survival, treatment response or any validated clinical category.

All patients, identifiers, studies, images and labels are fictional and procedurally generated by `demo_repo/data_gen.py`. No real patient records, personal information, client data, social-media data or public datasets are generator inputs. The CT-like and PET-like arrays are lightweight phantoms, not real scans, calibrated imaging or clinically validated volumes. This protocol states requirements; it does not report results.

2. Invariants

These invariants are normative. Every training and evaluation run in this repository must satisfy them.

- **INV-1 Patient independence** (`patient_independence`). No `patient_id` may appear in both the training set and the test set. A patient may contribute several slices; grouping applies across all of that patient's studies, slices and derived samples.
- **INV-2 Train-only fitting** (`train_only_fitting`). Any preprocessing statistic, normalization or learned transform must be fitted on training rows only.
- **INV-3 Inference consistency** (`inference_consistency`). Inference must apply the same deterministic preprocessing as evaluation.

An evaluation that violates INV-1 is contaminated: its metrics must not be described as independent-patient performance.

3. Dataset

The machine-readable specification is `configs/data_generation.json`; its numerical rules, mask and translation rules and RNG draw order are normative for generator 1.1.0.

The cohort has 200 fictional patients, one study and eight related slices each: 1,600 samples. Each sample is one `.npy` file holding a C-order float32 array of shape (2, 64, 64), loaded with `allow_pickle=False`. Channel 0 is CT-like and channel 1 is PET-like. The channels share anatomy, lesion coordinates and slice position. Values are finite and in [0, 1]; there is no Hounsfield-unit or SUV calibration.

Randomness uses NumPy Generator (PCG64). The data seed is 20260925, with an independent stream per patient from `SeedSequence([data_seed, patient_index])`. The label seed is 20260926: 100 zero and 100 one labels are permuted and assigned to ascending patient indices. Each patient has one label shared by all of its slices. These counts and parameters are engineering choices, not clinical statistics.

Anatomy and a persistent patient-specific texture are drawn independently of the label. Lesion size and intensity carry the simulated phenotype signal. Slices of one patient keep that patient's texture and add small translations and independent noise. Intensity mappings are fixed constants; no cohort statistic is used.

Disclosure: the persistent patient texture is an engineered correlation that makes individual patients recognizable across their slices. It exists to make evaluation defects measurable and is not evidence about real NSCLC data.

Metadata columns are `slice_id`, `patient_id`, `study_id`, `slice_index`, `array_path`, `label`, `generator_version`. `slice_id` is unique and is the DataFrame index. Filenames are identity-based and never encode labels. Model inputs are image values only, never identifiers, paths, seeds, labels or latent generation parameters.

4. Evaluation split

Each evaluation uses one two-way split of the complete 1,600-row dataset through the interface `make_split(df, seed, test_size=0.2)`, where `df` is the metadata sorted by `patient_id` and `slice_index`. The function returns two collections of index labels (train, test); consumers select rows with `.loc` and never treat labels as positions.

The split must satisfy INV-1. In addition, every returned partition must be valid before any metric is computed:

- row identifiers are unique and every row has a non-missing `patient_id`;
- both sets are non-empty and every returned identifier exists in the metadata;
- neither set contains duplicate identifiers and no row appears in both sets;
- together the two sets cover every row exactly once.

An invalid partition is a failed evaluation, never a zero-overlap success. No rows are silently removed. Label stratification is optional; observed class counts are recorded for every split, and seeds are never changed to obtain a favorable class balance.

5. Model and preprocessing

The model is a CPU-only `KNeighborsClassifier` with `n_neighbors=5`, `weights='distance'`, `algorithm='brute'`, `metric='euclidean'`, `n_jobs=1`. It is deterministic and has no random seed. There is no hyperparameter search.

Features are computed per image: for each channel, the mean of each non-overlapping 8x8 pixel block, giving a (2, 8, 8) grid flattened in C order to 128 features. This transform is fixed and not fitted on the cohort. No PCA, scaling, learned normalization, augmentation or CNN is part of this protocol version. The classifier is fitted on training rows only and predicts the probability of class label 1.

6. Metrics

The primary metric is patient-level ROC AUC. For each held-out patient, the score is the arithmetic mean of that patient's held-out slice probabilities only, paired with the patient's label. Training-slice predictions are never included.

The secondary metric is slice-level ROC AUC over held-out rows, reported separately. If the evaluated set has fewer than two label classes, the AUC is unavailable: it is stored as null with a reason, never as zero. If the training set has only one class, model evaluation is recorded as unavailable. AUC is not accuracy.

For every run, record: partition validity; train and test row and patient counts; class counts by rows and by patients; the number of patients present in both sets and that number divided by the number of unique test patients; both AUC values; timings; and the run ID.

7. Seeds and reporting

Seed 0 is the primary seed. Seeds 1 to 4 are predetermined sensitivity runs. Every result is retained, including failed and unavailable runs. For each evaluated configuration, report each seed's AUC and the mean and population standard deviation of the defined values with the defined/total count. When two configurations are compared, the per-seed difference is summarized only over seeds where both values are defined. Seed variability is not a confidence interval, and predictions are never pooled across seeds.

Different split procedures select different held-out patients, so an AUC difference between them describes this engineered benchmark only; it is not a causal estimate or a clinical performance claim. Zero or reversed differences are reported as observed. There is no target AUC.

8. Provenance and freeze

Before any evaluation that is reported, the configuration, generator, dependency lockfile, this protocol (Markdown and matching PDF), metadata and sorted image manifest are committed and frozen. Generator provenance is the clean commit that produced the bytes; a later commit adds the generated metadata and manifests.

The image manifest is a JSON array sorted by relative POSIX `array_path`, holding each file's path and the SHA-256 of its bytes. `data/generation_manifest.json` records the generator version and commit SHA, configuration, metadata and image-manifest hashes, dependency versions and sample count. Hashes are never invented. Regenerated arrays must match the manifest; any mismatch stops training. Training verifies every referenced image before recording the manifest hash.

Each run exports `run_artifacts/<run_id>/manifest.json` with the clean code SHA, dependency versions, configuration, metadata and verified image-manifest hashes, the split function with its code binding and settings, row and patient assignments, metrics, timings with their stated boundary, execution status and run ID. Local MLflow output alone is not sufficient evidence.

The dataset, model and metrics defined here do not change within protocol version 2.0. Any change to them requires a new version and a transparent rerun of every reported configuration.

9. Scope and limitations

The benchmark has no real population, no demographic or fairness assessment and no external clinical validation. Modality alignment, imaging realism and clinical plausibility are not validated. Results from this benchmark cannot establish legal or regulatory compliance and must not be described as compliant, safe or as permission to deploy.